
Two Coding Agents Optimize GraphViz: A Controlled Comparison

Ryan Tran¹, Hailu Xu¹

¹California State University, Long Beach
hailu.xu@csulb.edu

Abstract

We ask whether a general-purpose coding agent, given no hint about where a program spends its time, can find and fix a real performance bottleneck in a mature C codebase. Two agents — Claude Code and Codex — were each given an identical checkout of GraphViz pinned to the same commit and the same open-ended instruction: make the dot layout program faster without changing its output. Neither was told which function, file, or phase to target, and neither had access to the evaluation harness or the benchmark graphs. Both agents independently identified crossing minimization as the dominant cost, and both produced output-preserving speedups under independent measurement: $1.47\times$ and $1.16\times$ end-to-end on a 5,000-node graph, against an empirically measured noise floor of 8.4%. Neither agent used parallelism; both pursued serial algorithmic and data-layout changes. On a 10,000-node graph the two diverge — one agent improves to $1.57\times$ while the other stays flat at $1.15\times$ — and the phase-level data explains why: the workload’s dominant phase shifts from crossing minimization to coordinate assignment as the graph grows, and only one agent had touched the latter. We report what we measured, state the ceiling that bounds it, and name what we did not verify.

1 Introduction

Coding agents are now routinely asked to modify real software. Most evaluations of that ability measure whether an agent can make a failing test pass [6] — a task with a clear oracle, where success is checkable by running the suite. Performance optimization has no such oracle. An agent asked to make a program faster must first decide *what* to make faster, and a plausible-looking change that helps nothing is indistinguishable from a real improvement unless someone measures carefully.

This paper asks a narrow version of that question. Given a mature C codebase, no hint about where time is spent, and an instruction only to make the program faster without changing its output, can a coding agent find a genuine bottleneck and fix it?

We use the dot layout engine from GraphViz [3] as the subject. It is a good testbed for three reasons. It is real, widely deployed software rather than a benchmark written for the occasion. Its layered layout pipeline [8, 5] has distinct phases whose costs can be measured separately, so a speedup can be attributed rather than merely observed. And it has a deterministic text output format, which makes “the output did not change” a checkable claim rather than a judgment call.

What we did. Two agents — Claude Code and Codex — each received a pristine checkout of GraphViz pinned to commit 23b06521 and an identical prompt naming no target function, file, or phase. The prompt explicitly forbade the shortcuts that would fake a win: caching layout results, special-casing inputs, and reducing the iteration counts that govern layout quality. Neither agent had access to the evaluation harness or the graphs it uses. We rebuilt each agent’s result ourselves and measured it against an untouched baseline.

What we found. Both agents independently identified crossing minimization — roughly half of baseline runtime on our 5,000-node graph — as the target, and both produced byte-identical layout output. Measured independently, they achieved $1.47\times$ and $1.16\times$ end-to-end. Neither used parallelism: both went after serial algorithmic and memory-layout changes instead.

The more interesting result appears at 10,000 nodes, where the two agents separate. One improves to $1.57\times$; the other is flat at $1.15\times$. The phase-level data explains the divergence. As the graph grows, the dominant cost shifts away from crossing minimization toward coordinate assignment, and only one agent had modified the code responsible for the latter. The agent that made the broader change tracked the workload; the one that optimized a single function did not.

Contributions.

- A controlled protocol for evaluating agent-driven optimization, with an empirical noise floor established by benchmarking byte-identical binaries against each other before any treatment is measured.
- Phase-resolved measurements of two agents’ work on the same codebase, with per-stage attribution and the Amdahl ceiling that bounds it.
- The observation that neither agent chose parallelism when the method was left open, and evidence from the ceiling arithmetic that on this workload that choice was correct.
- A complete artifact: pinned source, both agents’ diffs and commit histories, raw timing data, and the figure-generation scripts.

Section 2 covers background. Section 3 describes the protocol in detail, including the null test. Section 4 reports measurements. Sections 5–6 interpret them and state what we did not verify.

2 Background and Related Work

The dot layout pipeline. GraphViz’s dot engine draws directed graphs using the layered approach introduced by Sugiyama et al. [8] and refined by Gansner et al. [5]. Layout proceeds in four phases, each depending on the last: *ranking* assigns nodes to layers, *crossing minimization* orders nodes within each layer to reduce edge crossings, *coordinate assignment* fixes x positions, and *spline routing* draws the edges. Ranking and coordinate assignment are both solved with network simplex; crossing minimization is an iterative heuristic. GraphViz exposes a `-Gphase=N` attribute that runs a prefix of this pipeline and stops, which we use for phase attribution (Section 3).

Agents modifying real code. Benchmarks such as SWE-bench [6] evaluate agents on resolving real repository issues, where a test suite provides the success signal. Performance work differs in that no test can tell you whether an agent found the *right* thing to optimize, and a change that is correct but useless passes every test. Prior work on learned compiler optimization [2] targets transformations chosen in advance; here the choice of target is the object of study.

Prior work on this codebase. An earlier study by some of the present authors applied a multi-model ensemble to the same GraphViz layout code and reported a $3.78\times$ speedup from OpenMP parallelization of crossing minimization [4]. That work has not been peer-reviewed. Our measurements do not reproduce it, and Section 5 gives the phase composition and Amdahl ceiling that bound what parallelizing that phase can yield on this workload. We note also that its reported prompt template specifies the target speedup as an input requirement, which makes the reported figure difficult to interpret as an independent measurement. The present paper shares only the codebase and the general question; its protocol, measurements, and conclusions are separate.

3 Method

The design goal was that source code be the *only* thing differing between the binaries we compare. Everything below follows from that.

3.1 Sandboxes and build

Three clones of the GraphViz repository were made, all checked out to commit 23b06521. One was kept untouched as the baseline; the other two were given to the agents. Each clone was placed on its own named branch so that the agents’ commit histories are preserved as artifacts rather than left on a detached HEAD.

All three were built with identical CMake flags (`-DCMAKE_BUILD_TYPE=Release`, Poppler and GVEdit disabled) and installed to separate prefixes. Each resulting dot binary resolves its libraries through an `@executable_path-relative rpath`, so a binary always loads its own build of the layout libraries. We verified this rather than assuming it.

3.2 The prompt

Both agents received the same text. It stated the goal — make dot faster on large graphs without changing its output — and deliberately named no function, file, or phase. Finding the hot code was part of the task under study. The prompt also:

- described the target workload shape (large layered graphs with heavy edge crossings) without supplying the evaluation graphs, so agents generated their own test inputs;
- required byte-identical `-Tdot` output and pointed at the 265 graphs in the repository’s own `tests/` directory;
- enumerated disallowed shortcuts: detecting or special-casing specific inputs, caching layout results across runs, reducing iteration counts or loosening convergence thresholds, hardcoding computed values, and modifying the test suite;
- stated that measurement would be independent, that identical builds vary by several percent on the target machine, and that a modest honest result or no result at all was an acceptable outcome.

The prohibition on reducing iteration counts matters most. Crossing minimization is iteration-bounded, so cutting its iteration limit produces a large, real, and reproducible speedup that is not an optimization. The prompt reframed that as a finding to report rather than a change to ship, giving an agent that noticed it somewhere honest to put it.

Agents worked autonomously with no turn or time limit and were asked to commit incrementally and to write a report describing what they profiled, what they changed, what failed, and what they verified. Neither agent had access to the evaluation harness, the benchmark graphs, or the other agent’s sandbox.

3.3 Measurement

All timings use `hyperfine 1.20.0` [7] with 10 warmup and 10 timed runs. Every binary for a given (graph, phase) is measured in a *single* `hyperfine` invocation, so the binaries run seconds apart under the same thermal conditions. This matters: measuring them in separate invocations produced order-dependent bias on our machine large enough to swamp the effects we are trying to detect. Consequently, only comparisons made within one invocation are meaningful, and we do not compare numbers across runs.

Phase attribution uses `-Gphase=N`, which runs a prefix of the pipeline and returns. Phase timings are therefore *cumulative*: phase 2 includes phase 1. Per-stage costs are obtained by differencing adjacent phases, which compounds the error of both terms; we report them as weaker evidence than the cumulative figures throughout.

Evaluation graphs are layered directed graphs generated by a fixed-seed script: 100 nodes per rank, each node with three edges to randomly chosen nodes in the next rank, at 5,000 and 10,000 nodes. The generator is deterministic — we verified that regenerating a graph reproduces it byte-for-byte — so the described test suite and the measured one cannot diverge.

Experiments ran on an Apple M4 Pro (12 cores: 8 performance, 4 efficiency), 24 GB unified memory, macOS 26.5.2, with Apple Clang 17.

3.4 The null test

Before measuring any agent’s work we established what “no difference” looks like. The baseline build tree was installed to three separate prefixes, producing three binaries that are byte-identical (SHA-1 b02f718d . . .); these were then benchmarked against one another exactly as the real binaries are, as three commands in one `hyperfine` invocation.

Any spread observed there is attributable to run ordering, thermal drift, and per-run variance — everything the real comparison measures except source code. Section 4 reports the result, and we use it as the floor below which no claimed speedup is meaningful. We rebuilt nothing for this test: a recompilation would embed a new build-date string into the version, introducing a difference rather than holding source constant.

3.5 Correctness checking

Correctness is checked on `dot -Tdot` output rather than a rendered image. The `dot` text format is deterministic, so byte-identity is a meaningful assertion, whereas image comparison would fold in rendering and floating-point behaviour we are not studying. We additionally read both agents’ diffs against the list of disallowed shortcuts in the prompt.

4 Results

4.1 Noise floor

Figure 1 shows the null test: three byte-identical binaries benchmarked against one another on the 5,000-node graph. Their means differ by 4.6%, 4.6%, 8.4%, and 4.2% in phases 1, 2, 3, and full respectively.

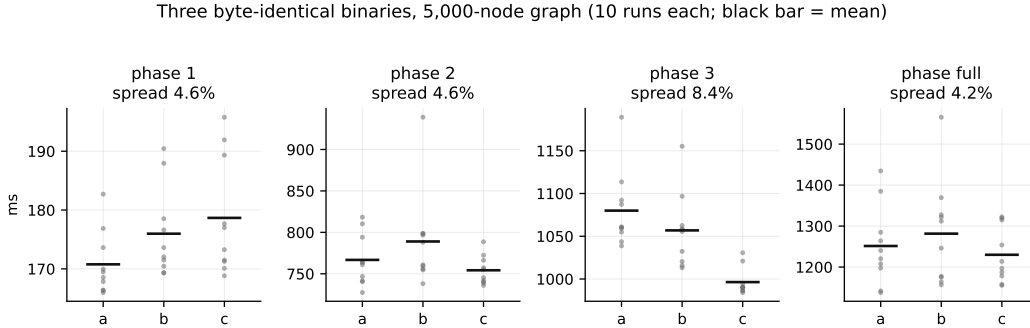


Figure 1: Three byte-identical binaries, 5,000-node graph, 10 runs each. All spread here is ordering, thermal, and per-run variance — the source code is the same program. The worst case, 8.4% in phase 3, is the floor we hold claimed speedups against.

The largest disagreement between identical programs is 8.4%, and we use that conservative figure rather than the 4.2% end-to-end value as the threshold for a meaningful result. Note that this is substantially larger than the within-command standard deviation, which is around 1.5% end-to-end; reporting only the latter would overstate confidence.

4.2 Where baseline time goes

Table 1 and Figure 2 give the baseline phase composition at both graph sizes. Parsing, measured separately with GraphViz’s `gc` tool, accounts for 11.9 ± 0.4 ms at 5,000 nodes — about 1% of runtime — and is included in the ranking figure below.

The shift is the important part. Crossing minimization falls from 49.3% to 37.7% of runtime as the graph doubles, while coordinate assignment rises from 22.6% to 48.7% and becomes dominant. Both are solved with network simplex in the coordinate case; the crossing phase is an iterative heuristic. This reweighting drives the divergence reported in Section 4.5.

Table 1: Baseline runtime composition, by differencing adjacent cumulative phases. Crossing minimization dominates at 5,000 nodes; coordinate assignment overtakes it at 10,000.

Stage	5,000 nodes		10,000 nodes	
	ms	share	ms	share
Ranking	198.8	17.8%	214.7	6.2%
Crossing minimization	549.9	49.3%	1312.5	37.7%
Coordinate assignment	252.5	22.6%	1697.6	48.7%
Spline routing	114.6	10.3%	261.2	7.5%
Total	1115.7	100%	3486.0	100%

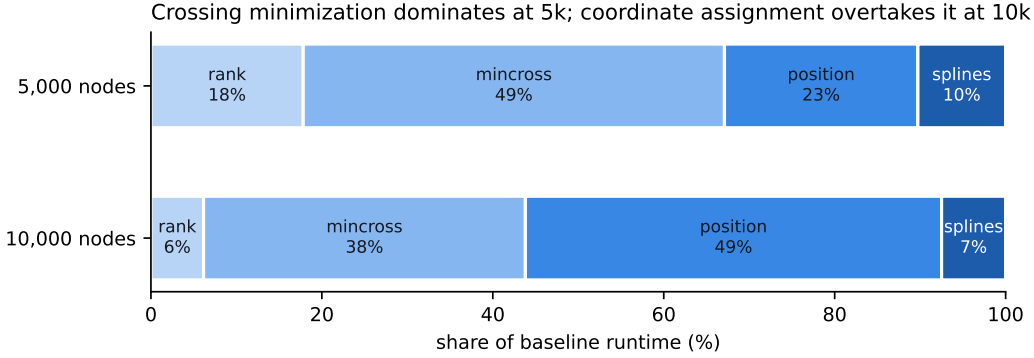


Figure 2: Baseline runtime composition at both graph sizes.

4.3 End-to-end results

Table 2 gives cumulative phase timings for all three binaries at both sizes, and Figure 3 plots the end-to-end result with every individual run shown.

Table 2: Cumulative phase timings, mean $\pm$ standard deviation over 10 runs. Phases are cumulative: each row includes everything above it. All three binaries were measured in one hyperfine invocation per row.

Graph	Phase	Baseline	Claude	Codex
5,000	1 (ranking)	198.8 $\pm$ 18.4	184.5 $\pm$ 14.9	173.9 $\pm$ 8.8
	2 (+ crossing min.)	748.7 $\pm$ 27.6	476.9 $\pm$ 23.2	572.9 $\pm$ 20.1
	3 (+ coordinates)	1001.2 $\pm$ 27.0	640.2 $\pm$ 23.7	840.2 $\pm$ 28.2
	full (+ splines)	1115.7 $\pm$ 16.2	758.9 $\pm$ 19.9	960.9 $\pm$ 27.1
10,000	1 (ranking)	214.7 $\pm$ 1.4	214.4 $\pm$ 2.4	221.1 $\pm$ 11.4
	2 (+ crossing min.)	1527.2 $\pm$ 20.0	978.3 $\pm$ 19.2	1245.9 $\pm$ 42.5
	3 (+ coordinates)	3224.8 $\pm$ 214.1	2109.1 $\pm$ 74.7	2720.6 $\pm$ 151.9
	full (+ splines)	3486.0 $\pm$ 263.4	2226.6 $\pm$ 26.2	3028.9 $\pm$ 292.3

At 5,000 nodes Claude reaches $1.47\times$ and Codex $1.16\times$ end-to-end. Both exceed the 8.4% floor, Claude by a wide margin. Codex’s 16% improvement is roughly twice the floor: real, but a margin worth stating explicitly rather than reporting as a bare ratio.

4.4 Per-stage attribution

Figure 4 shows every run at 5,000 nodes. The ranking phase is the cautionary case: the three binaries lie within about 25 ms of one another on a measurement whose per-binary standard deviation is 9–18 ms, and their distributions overlap almost completely. We report ranking as no difference.

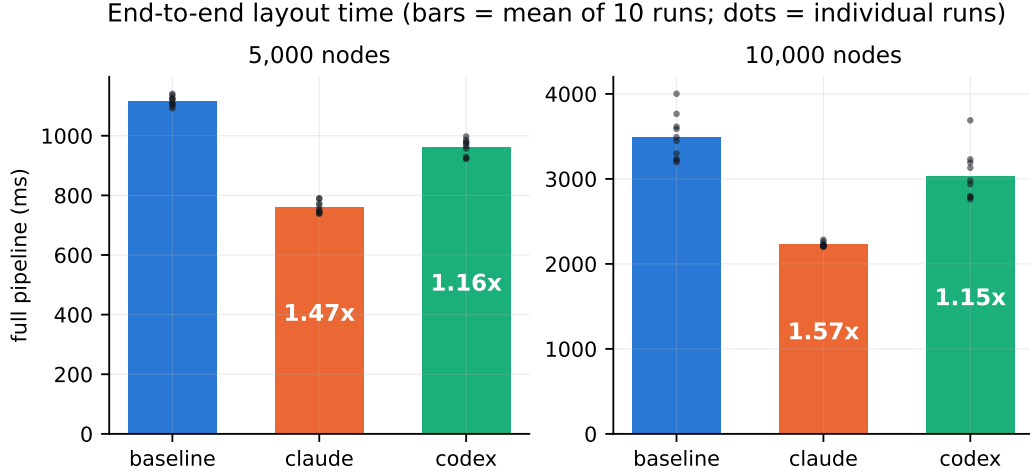


Figure 3: End-to-end layout time. Bars are means of 10 runs; dots are the individual runs.

Table 3: Per-stage speedup over baseline, derived by differencing adjacent phases. Differencing compounds the error of both terms; these are weaker evidence than Table 2.

Stage	5,000 nodes		10,000 nodes	
	Claude	Codex	Claude	Codex
Ranking	1.08×	1.14×	1.00×	0.97×
Crossing minimization	1.88×	1.38×	1.72×	1.28×
Coordinate assignment	1.55×	0.94×	1.50×	1.15×
Spline routing	0.97×	0.95×	—	—

Neither agent modified the ranking path in a way that would be expected to matter here, since the generated graphs pin ranks explicitly and leave little ranking work to do.

Spline routing at 10,000 nodes is omitted from Table 3. It is derived by subtracting two quantities whose standard deviations are 214 ms and 263 ms, and the resulting figure is not meaningful.

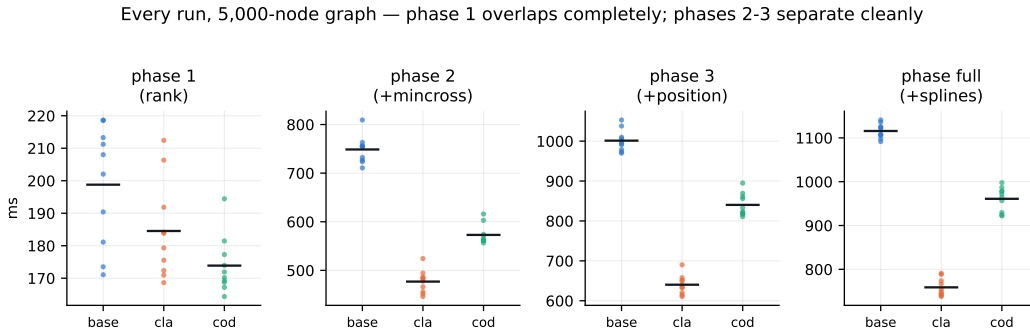


Figure 4: All 10 runs per binary per phase, 5,000-node graph. The ranking phase overlaps completely across all three binaries; phases 2 and 3 separate cleanly.

4.5 Scaling

Doubling the graph separates the two agents (Figure 5). Claude improves from 1.47× to 1.57×; Codex is flat, 1.16× to 1.15×. Table 3 shows the mechanism: at 10,000 nodes coordinate assignment is 48.7% of runtime, and Claude achieves 1.50× on it against Codex’s 1.15×.

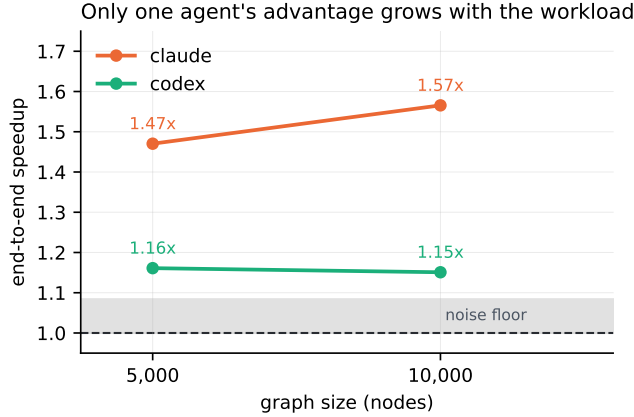


Figure 5: End-to-end speedup against graph size, with the noise floor shaded. Two points are suggestive, not a fitted trend.

4.6 What the agents changed

Both agents modified crossing minimization, and both arrived independently at the same core idea: the two symmetric crossing-count computations for an adjacent node pair can be fused into a single pass, since any pair of incident edges contributes to at most one of the two counts.

Codex made two code commits touching only `lib/dotgen/mincross.c` (+122/−17 lines). Beyond the fused counts, it replaced a linear prefix scan in the crossing-count routine with a Fenwick tree, reducing that inner loop from $O(n)$ to $O(\log n)$.

Claude made eight code commits touching `lib/dotgen/mincross.c` and `lib/common/ns.c` (+978/−169 lines). It found the same fused-count transformation, then restructured the network simplex implementation: the depth-first traversal keeps its current frame in local variables backed by a reusable stack, and the scattered per-node fields reached through accessor macros were replaced by a dense 32-byte-per-node state array with cached neighbour pointers. This is a memory-locality change, and it is what produces the coordinate-assignment improvement in Table 3.

4.7 Correctness

Both agents’ binaries produce `dot -Tdot` output byte-identical to the baseline on the evaluation graphs (SHA-1 3560c5d0... at 5,000 nodes). We read both diffs against the prompt’s list of disallowed shortcuts and found no reduced iteration counts, early exits, input special-casing, or modified tests. Neither diff contains any OpenMP directive, thread creation, or other concurrency construct.

5 Discussion

5.1 Both agents found the bottleneck

Neither agent was told where to look, and both went to crossing minimization — the single largest consumer of runtime on the 5,000-node workload at 49.3%. Both further converged on the same specific transformation, fusing the two symmetric crossing-count passes, despite working in isolation. On this evidence the bottleneck-finding half of the task is not the hard part.

The two diverge in how far they went. Codex stopped after optimizing the phase it had identified. Claude continued into network simplex, which is used for both ranking and coordinate assignment, and that decision is what its lead consists of.

5.2 The ceiling, and why breadth mattered

Amdahl’s law [1] bounds what optimizing a single phase can yield. From the composition in Table 1, driving crossing minimization to *zero* cost caps end-to-end speedup at $1.97\times$ at 5,000 nodes — and at only $1.60\times$ at 10,000, because that phase is a smaller fraction of the larger workload.

That second number is the useful one. Claude measures $1.57\times$ at 10,000 nodes, which is essentially at the ceiling for crossing minimization alone. It can sit there only because it also improved coordinate assignment; no amount of additional work on crossing minimization could have taken it past $1.60\times$. Codex, having modified one file, is bounded by a ceiling that shrinks as the workload grows.

The general point is that on a multi-phase pipeline, the value of an optimization depends on how the phase weights move with problem size. An agent that optimizes the currently-dominant phase captures a ceiling that may be receding. We would not have observed this with a single graph size, and we suggest that phase-resolved measurement at more than one scale should be standard in this kind of evaluation.

5.3 Neither agent chose parallelism

Given a free choice of method, both agents pursued serial algorithmic and data-layout changes. Neither diff contains a single OpenMP directive or thread.

The ceiling arithmetic suggests this was correct rather than merely conservative. Assume *perfect* linear scaling across the 8 performance cores — an assumption crossing minimization’s rank-by-rank sweeps would not support. Parallelizing that phase alone then caps end-to-end speedup at $1.76\times$ at 5,000 nodes and at $1.49\times$ at 10,000, the ceiling falling as the phase’s share of runtime does.

At 5,000 nodes Claude’s serial $1.47\times$ is below that bound. At 10,000 it is above it: $1.57\times$ measured, against $1.49\times$ for idealized eight-core parallelization of the phase both agents targeted. A serial change outperformed what perfect parallelization of the nominated target could have achieved — without introducing concurrency, and therefore without introducing the possibility of a data race.

This bears on the prior work discussed in Section 2, which framed the problem as an OpenMP parallelization task and reported $3.78\times$. That figure exceeds the $3.56\times$ ceiling for eliminating *both* of the two dominant phases entirely at 5,000 nodes, and is roughly twice what perfect parallelization of its stated target could produce. We make no claim about how that number arose; we observe that our measurements of the same codebase do not support it, and that the phase composition bounds it.

5.4 Cost

The two runs differed substantially in resource consumption. Claude ran for 4 h 27 m of working time and consumed roughly 9.2×10^7 tokens; Codex ran 23 m 42 s wall-clock and consumed roughly 5.1×10^6 . That is more than an order of magnitude in both dimensions for a $1.27\times$ difference in delivered layout time at 5,000 nodes, widening to $1.36\times$ at 10,000.

We report these figures but caution against reading them as a like-for-like comparison. The two systems account for tokens differently — one folds cached input into its input total, the other reports them separately — and the time figures are not the same measurement, one excluding time spent waiting on user input and the other being raw elapsed time. Whether the additional effort was *necessary* for the additional result, or merely accompanied it, is not something a single run of each can answer.

6 Limitations

We state these plainly because several of them bound the conclusions above.

One run per agent. Each agent was invoked once. Agent behaviour is stochastic, so we cannot say whether the ordering we observed is stable or whether a second invocation of either would find a different target. This is the single weakest point in the study, and it means our results describe two particular runs rather than two systems.

One graph topology. All measurements use layered graphs with uniform rank width and constant fanout. Real inputs to dot vary in density, depth, and regularity, and the phase composition that drives our central argument is a property of this generator. Two graph sizes is enough to show that composition shifts; it is not enough to characterize how.

Correctness is verified narrowly. Byte-identical output was confirmed on the evaluation graphs. We did *not* independently verify output identity across the 265 graphs in the GraphViz test suite, nor did we run the project’s regression suite against the modified builds. Both agents report having done so; we did not confirm it.

No race detection. We did not run ThreadSanitizer against any binary. Since neither agent introduced concurrency this is a smaller gap than it would otherwise be, but it means our correctness claim rests on output identity and diff inspection rather than on tooling. We note that the sanitizer runtimes shipped with the host platform do not currently execute at all — an empty program built with `-fsanitize=thread` crashes during sanitizer initialization on macOS 26 — so any such check requires a Linux container, which we did not run.

Derived stage figures. Per-stage costs come from differencing cumulative phase measurements, which compounds the error of both terms. Where that error is large relative to the difference — ranking at both sizes, spline routing at 10,000 nodes — we report no result rather than a number.

Cost figures are not comparable. As described in Section 5, the token and time accounting differs between the two systems in ways we cannot normalize.

Single machine. All measurements come from one Apple M4 Pro. The noise floor, the phase composition, and the memory-locality effect that drives Claude’s coordinate assignment result are all plausibly machine-dependent.

Reproducibility Statement

Every number in this paper comes from data in the artifact below. No result is estimated, extrapolated, or reported from a source other than the raw measurement files.

Artifacts.

- Claude’s diff and commit history:
<https://github.com/harvest7777/claude-graphviz-run/compare/main...agent-work>
- Codex’s diff and commit history:
<https://github.com/harvest7777/codex-graphviz-run/compare/main...agent-work>
- Raw timing data and session dumps:
<https://github.com/harvest7777/graphviz-results-dump>

Source. GraphViz at commit 23b06521ab916e7f4f2ae2f9ef8413a8ccc66601, from <https://gitlab.com/graphviz/graphviz.git>. Both agent branches are named agent-work and branch from that commit.

Build.

```
cmake -S <sandbox> -B <build> \  
  -DCMAKE_INSTALL_PREFIX=<prefix> \  
  -DWITH_POPPLER=OFF -DWITH_GVEDIT=OFF \  
  -DCMAKE_BUILD_TYPE=Release  
cmake --build <build> -j12 && cmake --install <build>
```

Identical for all three sandboxes. On macOS a Bison ≥ 3.0 path must be passed explicitly, as the system Bison is too old for the GraphViz build.

Measurement. hyperfine 1.20.0, 10 warmup and 10 timed runs, all binaries for a given (graph, phase) in a single invocation. Graphs are produced by the fixed-seed generator in the artifact repository. Raw per-run timings for every configuration — including the null test — are in the results repository as hyperfine JSON exports.

Figures. Every figure is regenerated from the raw JSON by a single script in the artifact (figures/make_figures.py).

Environment. Apple M4 Pro, 12 cores (8 performance, 4 efficiency), 24 GB unified memory, macOS 26.5.2, Apple Clang 17.0.0.

Agents. claude-opus-5 (Claude Code) and gpt-5.6-terra (Codex), each invoked once, autonomously, with the prompt reproduced in the artifact repository. Neither had network access to the evaluation harness or benchmark graphs.

7 Conclusion

Two coding agents, given an identical checkout of GraphViz and a prompt naming no target, both located the dominant cost in the dot layout pipeline and produced output-preserving speedups: $1.47\times$ and $1.16\times$ end-to-end at 5,000 nodes, against an 8.4% measured noise floor. Finding the bottleneck was evidently not the difficult part of the task.

What separated them was scope. At 10,000 nodes the dominant phase shifts from crossing minimization to coordinate assignment, and the agent that had restructured network simplex — code shared by both phases — improves to $1.57\times$ while the agent that optimized a single function stays flat at $1.15\times$. An optimization’s value depends on whether the phase it targets grows or shrinks with the workload, and a single problem size cannot reveal that.

Neither agent used parallelism. On this workload the ceiling arithmetic suggests that was the right call: at 10,000 nodes, idealized eight-core parallelization of crossing minimization caps end-to-end speedup at $1.49\times$, and one agent exceeded that serially at $1.57\times$ — without introducing concurrency or the possibility of a race.

The results are modest, and we have tried to report them at the size they actually are. The protocol may be the more durable contribution: measure the noise floor with byte-identical binaries before measuring anything else, hold the evaluation harness out of the agents’ reach, forbid the shortcuts that manufacture speedups, and attribute results by phase at more than one problem size. Each of these caught something in this study that a simpler design would have reported as a finding.

Immediate future work is to repeat both agent runs to establish whether the ordering is stable, extend to a third graph size and to non-layered topologies, and complete the correctness checks named in Section 6.

References

- [1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.
- [2] Chris Cummins, Volker Seeker, Dejan Grubisic, Mostafa Elhoushi, Youwei Liang, Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Kim Hazelwood, Gabriel Synnaeve, and Hugh Leather. Large language models for compiler optimization, 2023. arXiv preprint.
- [3] John Ellson, Emden Gansner, Lefteris Koutsofios, Stephen C. North, and Gordon Woodhull. Graphviz — open source graph drawing tools. In *Graph Drawing (GD 2001)*, LNCS 2265, pages 483–484, 2001.
- [4] Edwyn Estrada, Hailu Xu, and Chunhua Liao. Learning to optimize: Automated parallel code generation for Graphviz dot layout algorithms, 2025. Unpublished manuscript. OpenReview submission <https://openreview.net/forum?id=88zyE3fJzQ>.
- [5] Emden R. Gansner, Eleftherios Koutsofios, Stephen C. North, and Kiem-Phong Vo. A technique for drawing directed graphs. *IEEE Transactions on Software Engineering*, 19(3):214–230, 1993.

- [6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [7] David Peter. hyperfine: A command-line benchmarking tool, 2023. Version 1.20.0. <https://github.com/sharkdp/hyperfine>.
- [8] Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiko Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125, 1981.